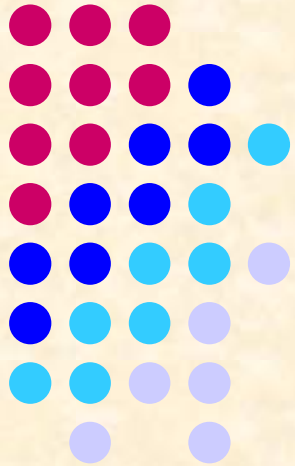
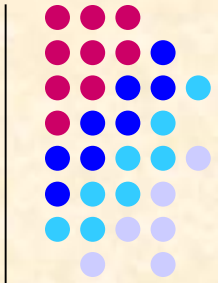
Langage SQL

Dr MOUSSE A. Mikael



Plan du cours



- Introduction
- Langage de Manipulation des Données (LMD)
- Langage de Définition des Données (LDD)
- Langage de Contrôle des Données (LCD)

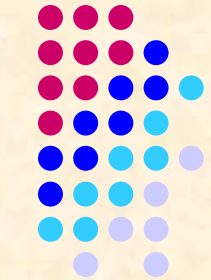
Définition

- **Structured Query Language (SQL)**
- Langage informatique standard et normalisé destiné à interroger ou manipuler une base de données relationnelle

Historique

- Modèle relationnel pour SGBD
  - (Codd,1970)
- SQUARE
  - Algèbre relationnelle ~ anglais
  - (Boyce, Chamberlin, King & Hammer, 1975)
- SEQUEL
  - *Structured English QUEry Language*
  - (Chamberlin, Astrahan, Eswaran, Chamberlin, Griffiths & Lorie, 1976)
- SQL1
  - Standard ANSI(1986) et ISO(1987)
  - Ajout des contraintes de référence en 1989 (clés primaire/étrangère)
- SQL2
  - Révision majeure : expressions régulières, requêtes récursives, déclencheurs,… (1992)
- SQL3
  - Fonctions orientées objet, manipulation XML,etc… (2003)

Logiciels implémentant SQL



- Access
- Advantage Database
- Base
- DB2
- Firebird
- FoxPro - Visual Foxpro
- HyperFile
- Informix
- Ingres
- InterBase
- MaxDB
- Microsoft SQL Server

Mimer
MySQL
Ocelot
Oracle
PostgreSQL
SmallSQL
SQL 2000 i (Pervasive)
SQLBase (Centura)
SQLite
SQL/MM
Sybase
Paradox
Windev

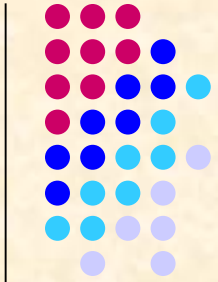
Subdivisions du SQL

- Data Definition Language (DDL)
  - ⇨ Définition et modification de la base de données
    - CREATE, ALTER, DROP
- Data Manipulation Language (DML)
  - ⇨ Manipulation des données de la base
    - SELECT, INSERT, UPDATE, DELETE
- Data Control Language (DCL)
  - ⇨ Gestion des utilisateurs de la base
    - GRANT, REVOKE

SQL1 Complet

- Transaction Control Language (TCL)
  - ⇨ Gestion des transactions (« paquets d'instructions »)
    - SET TRANSACTION, COMMIT, ROLLBACK
- SQL Procédural
  - ⇨ Eléments procéduraux pour interfaçage avec autres languages
    - PSM (*Persistent Stored Module*), CLI (*Call Level Interface*), *Embedded SQL*,…

Implémentations du SQL

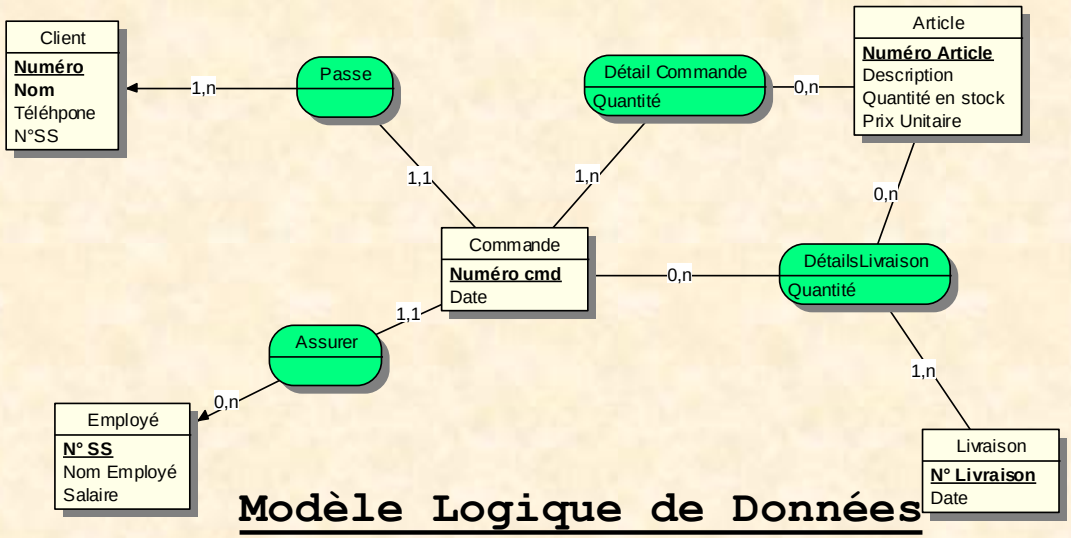


- SQL est une norme, mais chaque éditeur a développé son propre dialecte
  - ⇨ Bases de données incompatibles
  - ⇨ Par rapport à la norme, certaines fonctions sont implémentées de manière différente, ou pas du tout, d'autres sont ajoutées
- Liste comparative :

  http://sql.developpez.com/sqlaz/fonctions/

Schéma utilisé comme exemple

Modèle Conceptuel de Données



Modèle Logique de Données

Client (noClient, Nom, Téléphone, N°SS)

Article (N° Article, Description, P.U., Quantité en stock)

Commande (N° Com, Date, *N° Client, N°SS_Employé*)

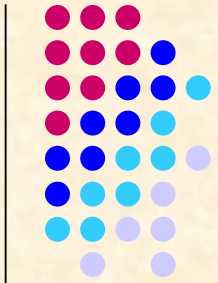
Détail commande (N° Com, N° Article, Quantité)

Employé (N°SS, Nom_Employé, Salaire)

Livraison (N°Livraison, Date)

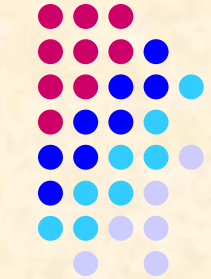
DétailsLivraison (N°Livraison, N°Cmd, N°Art, Quantité)

Langage de Manipulation des Données (LMD)



- SELECT
- Projection
- Sélection
- Tri
- Conflit de noms d’attributs
- Jointures
- Opérateurs ensemblistes
- Fonctions
- Regroupements
- Requêtes imbriquées
- Conditions ALL,ANY,EXISTS
- Modification de données

Commande de base : SELECT



- Syntaxe

SELECT [DISTINCT | ALL] <clause1>[<clause2>…]
FROM <nom_table1> [, <nom_table2>,…]
[WHERE <condition1> [AND|OR <condition2> …]]
[GROUP BY <regroupement1>[<regroup2>…]]
[HAVING condition]
[ORDER BY liste de colonnes]

Composé de 6 parties, dont 2 sont obligatoires

SELECT minimal

Afficher l'ensemble des commandes

ATTENTION, comme vous sélectionnez tous les attributs de la table, il n'y a pas de projection

Commande

```
SELECT      *
FROM        Commande
```

Table *Commande*			
N° Com	**Date**	**N° Client**	**N°SS Emp**
1	01/06/2000	10	123
2	02/06/2000	20	456
3	02/06/2000	10	123
4	05/07/2000	10	456
5	09/07/2000	30	789
6	09/07/2000	20	789
7	15/07/2000	40	789
8	15/07/2000	40	123

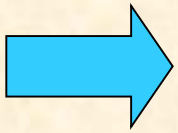
Projection

Quels sont pour chaque commande, le numéro du client et la date de la commande ?

$$\pi_{noClient,\ dateCommande}(Commande)$$

```
SELECT      noClient, dateCommande
FROM        Commande
```

Table *Commande*			
N° Com	**Date**	**N° Client**	**N°SS Emp**
1	01/06/2000	10	123
2	02/06/2000	20	456
3	02/06/2000	10	123
4	05/07/2000	10	456
5	09/07/2000	30	789
6	09/07/2000	20	789
7	15/07/2000	40	789
8	15/07/2000	40	123



noClient	dateCommande
10	01/06/2000
20	02/06/2000
10	02/06/2000
10	05/07/2000
30	09/07/2000
20	09/07/2000
40	15/07/2000
40	15/07/2000

PROJECTION sans éliminer les doublons

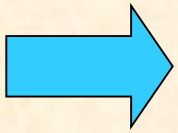
Projection

Quels sont pour chaque commande, le numéro du client et la date de la commande ?

$$\pi_{noClient,\ dateCommande}(Commande)$$

```
SELECT    DISTINCT noClient, dateCommande
FROM      Commande
```

Table *Commande*			
N° Com	Date	N° Client	N°SS Emp
1	01/06/2000	10	123
2	02/06/2000	20	456
3	02/06/2000	10	123
4	05/07/2000	10	456
5	09/07/2000	30	789
6	09/07/2000	20	789
7	15/07/2000	40	789
8	15/07/2000	40	123



noClient	dateCommande
10	01/06/2000
20	02/06/2000
10	02/06/2000
10	05/07/2000
30	09/07/2000
20	09/07/2000
40	15/07/2000

Sélection

Quels sont les *Articles* de numéro supérieur à 30 dont le prix est inférieur à 20 Francs ?

Algèbre relationnelle : $\sigma_{prixUnitaire < 20.00 \text{ ET } noArticle > 30}(Article)$

```
SELECT      *
FROM        Article
WHERE       prixUnitaire < 20 AND noArticle > 30
```

noArticle	description	prixUnitaire	Quantité
60	Erable argenté	15.99	10
70	Herbe à puce	10.99	10
95	Génévrier	15.99	10

Sélection

Afficher les *Articles* dont le numéro est supérieur à 30 et le prix est inférieur à 20 Francs, ou dont le numéro est inférieur à 20 et le prix supérieur à 30 Francs

```
SELECT     *
FROM       Article
WHERE      prixUnitaire < 20 AND noArticle > 30
OR prixUnitaire > 30 AND noArticle < 20
```

AND est plus fort que OR

On peut utiliser les parenthèses pour définir la priorité de résolution des conditions

```
SELECT     *
FROM       Article
WHERE      (prixUnitaire < 20 AND noArticle > 30)
OR (prixUnitaire > 30 AND noArticle < 20)
```

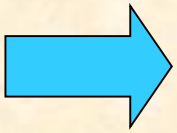
Sélection

Afficher les *Commandes* du mois de juin de l'année 2000

```
SELECT      *
FROM        Commande
WHERE       dateCommande BETWEEN 01/06/2000 AND 30/06/2000
```

```
SELECT      *
FROM        Commande
WHERE       dateCommande >= 01/06/2000 AND dateCommande <=30/06/2000
```

Table *Commande*			
N° Com	**Date**	**N° Client**	**N°SS Emp**
1	01/06/2000	10	123
2	02/06/2000	20	456
3	02/06/2000	10	123
4	05/07/2000	10	456
5	09/07/2000	30	789
6	09/07/2000	20	789
7	15/07/2000	40	789
8	15/07/2000	40	123



N° Com	**Date**	**N° Client**	**N°SS Emp**
1	01/06/2000	10	123
2	02/06/2000	20	456
3	02/06/2000	10	123

Sélection

Afficher les *Commandes* des *Clients* de *numéro* 10, 40 et 80

```
SELECT     *
FROM       Commande
WHERE      noClient = 10 OR noClient = 40 OR noClient = 80
```

```
SELECT     *
FROM       Commande
WHERE      noClient IN (10; 40; 80)
```

Table *Commande*			
N° Com	Date	N° Client	N°SS Emp
1	01/06/2000	10	123
3	02/06/2000	10	123
4	05/07/2000	10	456
7	15/07/2000	40	789
8	15/07/2000	40	123

Sélection

Afficher les *Clients* dont le nom commence par *D*

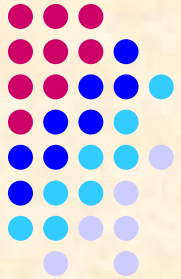
```
SELECT *
FROM      Client
WHERE     nomClient LIKE "D*"
```

N°	Nom_Client	Téléphone	N°SS Emp
20	Dollard Cash	(888)888-8888	456
80	Dollard Cash	(333)333-3333	123

Afficher les *Clients* dont la 3ème lettre du *nom* est *L* et dont la dernière lettre du nom est un *h*

```
SELECT *
FROM      Client
WHERE     nomClient LIKE "??L*h"
```

Sélection



Afficher les *Clients* dont le numéro de téléphone a été saisi

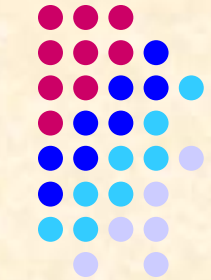
```
SELECT      *
FROM        Client
WHERE       Téléphone IS NOT NULL
```

N°	Nom_Client	Téléphone	N°SS Emp
10	Hugh Paycheck	(999)999-9999	123
20	Dollard Cash	(888)888-8888	456
30	Ye San Le Sou	(777)777-7777	123
40	Le Comte Hasek	(666)666-6666	456
50	Hafedh Lajoie	(555)555-5555	789
60	Comtesse Hasek	(666)666-6666	789
70	Coco McPoulet	(444)444-4419	789
80	Dollard Cash	(333)333-3333	123

expression **IS [NOT] NULL**

IS NULL est vraie si et seulement si la valeur associée est absente (valeur nulle)

Sélection et projection



Afficher les *noClient* et *dateCommande* des *Commandes* saisies après le *05/07/2000*

$$\pi_{noClient,\ dateCommande}\ (\sigma_{dateCommande\ >\ 05/07/2000}\ (Commande))$$

```
SELECT      noClient, dateCommande
FROM        Commande
WHERE       dateCommande > 05/07/2000
```

noClient	dateCommande
30	09/07/2000
20	09/07/2000
40	15/07/2000
40	15/07/2000

Projection avec Alias

- On peut utiliser des alias pour changer l'affichage d'un attribut

$\pi_{nom, Téléphone, \rho \text{ N°SS} \rightarrow \text{[Num sécurité Sociale]}}(Client)$

```
SELECT    Nom, Téléphone, N°SS AS [Num Sécurité Sociale]
FROM      Client
```

Nom_Client	Téléphone	Num sécurité sociale
Hugh Paycheck	(999)999-9999	951
Dollard Cash	(888)888-8888	753
Ye San Le Sou	(777)777-7777	486
Le Comte Hasek	(666)666-6666	197
Hafedh Lajoie	(555)555-5555	123
Comtesse Hasek	(666)666-6666	164
Coco McPoulet	(444)444-4419	188
Dollard Cash	(333)333-3333	133

ATTENTION,

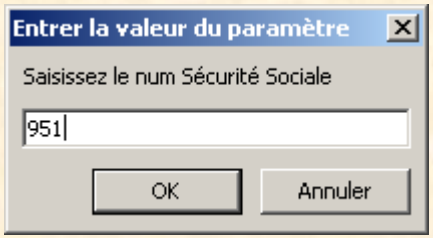
Sous Access, le nouveau nom doit être entre crochets

Requête paramétrée

Afficher le téléphone et le nom du client dont le numéro INSEE est **saisi par *l'utilisateur***

$$\pi_{nom,\ Téléphone}\left(\sigma_{\text{N°SS = [Saisissez le Num sécurité Sociale]}}(Client)\right)$$

```
SELECT     Nom, Téléphone, N°SS AS [Num sécurité sociale]
FROM       Client
WHERE N°SS = [Saisissez le Num Sécurité Sociale]
```



Nom_Client	Téléphone	Num sécurité sociale
Hugh Paycheck	(999)999-9999	951

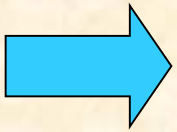
Tri du résultat

Afficher le *nom* et le *téléphone* des *Clients* dont le numéro est différent de 30 *par ordre alphabétique*

```
SELECT      Nom, Téléphone
FROM        Client
WHERE       Numéro <> 30
ORDER BY NOM
```

```
SELECT      Nom, Téléphone
FROM        Client
WHERE       Numéro <> 30
ORDER BY NOM ASC, Téléphone ASC
```

Nom	Téléphone
Coco McPoulet	(444)444-4419
Comtesse Hasek	(666)666-6666
Dollard Cash	(888)888-8888
Dollard Cash	(333)333-3333
Hafedh Lajoie	(555)555-5555
Hugh Paycheck	(999)999-9999
Le Comte Hasek	(666)666-6666



Nom	Téléphone
Coco McPoulet	(444)444-4419
Comtesse Hasek	(666)666-6666
Dollard Cash	(333)333-3333
Dollard Cash	(888)888-8888
Hafedh Lajoie	(555)555-5555
Hugh Paycheck	(999)999-9999
Le Comte Hasek	(666)666-6666

Conflit de noms d'attributs

- Deux techniques :
  - ⇨ Préfixage
    - Nom_table . nom_colonne

```
SELECT Client.Nom,…,
FROM Client, …
```

  - ⇨ Renommage (si nom long, ou auto-jointure)

```
SELECT    Cl.noClient, nomClient, noTéléphone, noCommande,
          dateCommande
FROM      Client AS Cl, Commande AS Co
WHERE     Cl.noClient = Co.noClient
```

```
SELECT    Client.noClient, nomClient, noCommande
FROM      Client AS Cl, Commande AS Co
WHERE     Cl.noClient = Co.noClient
```

Produit Cartésien

R1 X R2

SQL 1

```
SELECT      *
FROM        R1, R2
```

SQL 2

```
SELECT      *
FROM R1 CROSS JOIN R2
```

Afficher toutes les combinaisons possibles de lignes de Client et de Commande

```
SELECT      *
FROM        Client, Commande
```

Thêta-produit (thêta-jointure)

Afficher toutes les combinaisons possibles de lignes de Client numéro 10 et de Commande

$$\pi_{Client.noClient,\ nomClient,\ noTéléphone,\ noCommande}\ (Client \underset{noClient=10}{\bowtie} Commande)$$

```
SELECT     Client.noClient, nomClient, noTéléphone, noCommande
FROM       Client, Commande
WHERE Client. noClient = 10
```

```
SELECT     Client.noClient, nomClient, noTéléphone, noCommande
FROM       Client JOIN Commande
ON Client.noClient = 10
```

```
SELECT     Client.noClient, nomClient, noTéléphone, noCommande
FROM       Client JOIN Commande
USING (Client.noClient = 10)
```

Jointure naturelle (*)

Afficher toutes les informations au sujet des *Clients* et de leurs *Commandes*

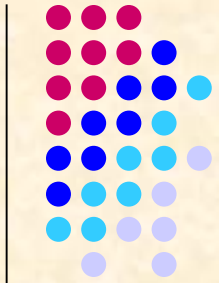
$$\pi_{Client.noClient,\ nomClient,\ noTéléphone,\ noCommande}(\sigma_{Client.noCliente = Commande.noClient}(Client \times Commande))$$

La jointure avec NATURAL JOIN est faite sur les colonnes de même nom (noClient)

```
SELECT     Client.noClient, nomClient, noTéléphone, noCommande
FROM       Client, Commande
WHERE      Client.noClient = Commande.noClient
```

```
SELECT     Client.noClient, nomClient, noTéléphone, noCommande
FROM       Client NATURAL JOIN Commande
```

Jointure naturelle (*)



```
SELECT     Client.noClient, nomClient, noTéléphone, noCommande
FROM       Client JOIN Commande ON
           Client.noClient = Commande.numeroClient
```

Quand les noms des colonnes sont différents, on peut utiliser JOIN avec une condition

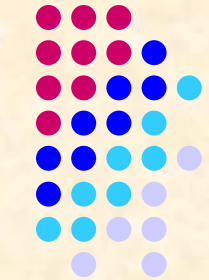
Jointure naturelle (*)

Afficher le nom des *Clients* qui ont passé des *Commandes le 01/07/2002*

```
SELECT     nomClient
FROM       Client Cl, Commande Co
WHERE      Cl.noClient = Co.noClient
AND Co.date = 01/07/2002
```

```
SELECT     nomClient
FROM       Client
WHERE      noClient IN
     (SELECT noClient
      FROM COMMANDE
      WHERE Date = 01/07/2002)
```

Jointure naturelle de plus de deux tables

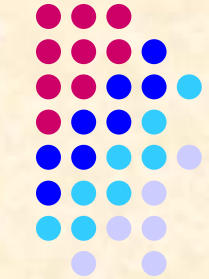


Afficher le *nom* des *Clients* qui ont commandé au moins un article *Imprimante*

$$\pi_{nomClient}(\sigma_{description = \text{“Imprimante”}}(Client * Commande * DétailsCommande * Article))$$

```
SELECT      nomClient
FROM        Client, Commande, DétailsCommande, Article
WHERE       description = 'Imprimante' AND
            Client.noClient = Commande.noClient AND
            Commande.noCommande = DétailsCommande.noCommande AND
            DétailsCommande.noArticle = Article.noArticle
```

Jointure naturelle de plus de deux tables



Afficher le *nom* des *Clients* qui ont commandé au moins un article *Imprimante*

$$\pi_{nomClient}(\sigma_{description = \text{“Imprimante”}}(Client * Commande * DétailsCommande * Article))$$

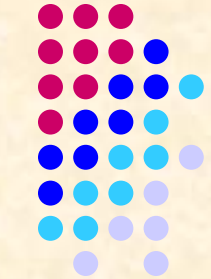
```
SELECT     nomClient
FROM       Client
WHERE noClient IN

           SELECT noClient
           FROM Commande
           WHERE noCommande IN

                 SELECT noCommande
                 FROM DétailsCommande
                 WHERE noArticle IN

                       SELECT noArticle
                       FROM Article
                       WHERE description = 'Imprimante'
```

Demi-Jointures



- Gauche

Afficher toutes les informations au sujet des *Clients* et de leurs *Commandes* et des *Clients* qui n'ont passé aucune commande

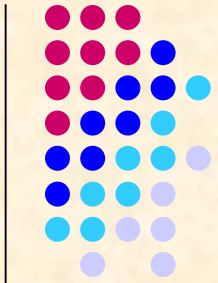
```
SELECT    Client.noClient, nomClient, noTéléphone, noCommande
FROM      Client LEFT OUTER JOIN Commande
```

- Droite

Afficher toutes les informations au sujet des *Clients* et de leurs *Commandes* et des *Commandes* dont le client n'est pas défini

```
SELECT    Client.noClient, nomClient, noTéléphone, noCommande
FROM      Client RIGHT OUTER JOIN Commande
```

Jointure Externe



Afficher toutes les informations au sujet des *Clients* et de leurs *Commandes,* des *Commandes* dont le client n'est pas défini, et des Clients qui n'ont pas de commande

```
SELECT    Client.noClient, nomClient, noTéléphone, noCommande
FROM      Client FULL OUTER JOIN Commande
```

Auto-jointure

Afficher les *Clients* qui ont le même numéro de téléphone

$$\pi_{Client.noClient,\ Client2.noClient,}\ (\sigma_{Client.noTéléphone = Client2.noTéléphone}\ (Client \times \rho_{Client2}\ (Client)))$$

```
SELECT     Client.noClient, Client2.noClient
FROM       Client, Client AS Client2
WHERE      Client.noTéléphone = Client2.noTéléphone
      AND Client.noClient <> Client2.noClient
```

```
SELECT     noClient, noClient2
FROM       Client JOIN Client AS Client2
                  ON Client.noTéléphone = Client2.noTéléphone
```

L'utilisation d'ALIAS est indispensable

Auto-jointure

$$\pi_{noClient,\ noClient2}\ (Client * \rho_{Client2(noClient2,\ nomClient2,\ noTéléphone,\ N°SS2)}\ (Client))$$

```
SELECT    noClient, noClient2
FROM      Client NATURAL JOIN
        Client AS Client2(noClient2, nomClient2, noTéléphone, N°SS2)
```

La jointure avec NATURAL JOIN est faite sur les colonnes de même nom

Auto-jointure

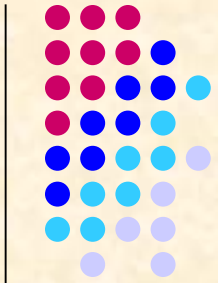
Afficher les *Employés* qui gagnent plus que l'employé "Chbeir"

```
SELECT     Employé.N°SS, CHBEIR.N°SS
FROM       Employé, Employé AS CHBEIR
WHERE      Employé.Salaire > CHBEIR.Salaire
       AND CHBEIR.Nom = "Chbeir"
```

```
SELECT     Employé.N°SS
FROM       Employé
WHERE      Salaire > (SELECT Salaire
                      FROM Employé
                      WHERE Nom = "Chbeir")
```

Une seule valeur doit être retournée

Opérateurs ensemblistes



- UNION ∪
- INTERSECT ∩
- MINUS –

Seul UNION est implémenté dans Access !

Les schémas des relations d'origine doivent être identiques (même nombre et mêmes types de colonnes)

Par défaut, les doublons sont éliminés (préciser ALL pour afficher les doublons)

Les noms des colonnes à l'affichage sont ceux du **premier SELECT**

UNION

Afficher le N°SS et le nom de toutes les personnes de la base, que ce soit des clients ou des employés

$$\pi_{\rho N°SS \rightarrow [N° INSEE],\ \rho nomClient \rightarrow [NomPersonne]}(\text{Client}) \cup \pi_{N°SS,nom_employé}(\text{Employé})$$

```
(SELECT    N°SS as N°INSEE, nomClient as nomPersonne
FROM       Client)
UNION ALL
(SELECT    N°SS, nom_Employé
FROM       Employé)
```

N° INSEE	NomPersonne
951	Hugh Paycheck
753	Dollard Cash
486	Ye San Le Sou
197	Le Comte Hasek
123	M. Barth
164	Comtesse Hasek
188	Coco McPoulet
133	Dollard Cash
456	M. Chbeir
789	Mme Kacimi
123	M. Barth

UNION

Afficher le N°SS et le nom de toutes les personnes de la base, que ce soit des clients ou des employés

```
(SELECT    N°SS as N°INSEE, nomClient as nomPersonne
FROM       Client)
UNION
(SELECT    N°SS, nom_Employé
FROM       Employé)
```

N° INSEE	NomPersonne
951	Hugh Paycheck
753	Dollard Cash
486	Ye San Le Sou
197	Le Comte Hasek
123	**M. Barth**
164	Comtesse Hasek
188	Coco McPoulet
133	Dollard Cash
456	M. Chbeir
789	Mme Kacimi

INTERSECT

Afficher le N°SS et le nom de tous les clients qui sont également employés

```
(SELECT    N°SS as N°INSEE, nomClient as nomPersonne
FROM       Client)
INTERSECT
(SELECT    N°SS, nom_Employé
FROM       Employé)
```

N° INSEE	NomPersonne
123	M. Barth

Par défaut, l'intersection est faite sur la position des colonnes

INTERSECT

Afficher le N°SS et le nom de tous les clients qui sont également employés

```
(SELECT    N°SS, nomClient as nomPersonne
FROM       Client)
INTERSECT CORRESPONDING BY (N°SS)
(SELECT    N°SS, nom_Employé
FROM       Employé)
```

N° INSEE	NomPersonne
123	M. Barth

L'intersection est faite sur la colonne N°SS

Fonctions

- Fonctions dépendantes du SGBD
  - Fonctions sur chaînes de caractères
  - Fonctions sur des entiers
  - Fonctions sur des dates

Exple : Fonctions intégrées dans Access

Afficher les *Commandes* de la journée

```
SELECT *
FROM       Commande
WHERE      dateCommande = Maintenant()
```

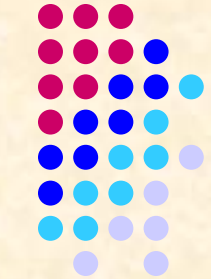
Fonctions

- Fonctions génériques
  - Fonctions arithmétiques
    - +, *, /, -
  - Fonctions ensemblistes
    - **MAX** : Fournit la valeur maximale
    - **MIN** : Fournit la valeur minimale
    - **COUNT** : Fournit la cardinalité d'un ensemble
    - **SUM** : Somme de toutes les valeurs
    - **AVG** : Moyenne de toutes les valeurs

Valeurs numériques, caractère et date

Valeurs Numériques

Fonctions Ensemblistes



- Elles peuvent être utilisées dans la clause SELECT ou dans la clause WHERE
- Elles agissent sur un ensemble de valeurs d'une colonne
- Elles donnent toujours une seule valeur
- Si l'argument est l'ensemble vide
  - ⇨ COUNT donne 0
  - ⇨ Les autres fonctions donnent NULL
- L'argument de ces fonctions ne peut pas être une fonction
  - ⇨ SELECT AVG(SUM(Salaire)) FROM … est invalide
  - ⇨ Mais on peut faire:
    - SELECT AVG(x) FROM (SELECT SUM(Salaire) AS x FROM ..))
- Les valeurs nulles sont éliminées avant l'application de la fonction, sauf pour COUNT(*)

Moyenne AVG

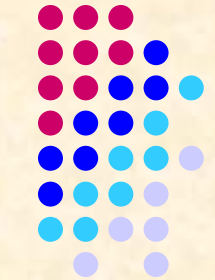
Afficher le salaire moyen des employés

```
SELECT AVG(Salaire)
FROM       Employé
```

Afficher les employés qui gagnent plus que le salaire moyen

```
SELECT Nom
FROM   Employé
WHERE Salaire >=(SELECT AVG(Salaire)
                 FROM EMployé)
```

Count(*)



- **COUNT(*)** compte toutes les lignes d'une table ou d'un résultat d'une sélection (sans éliminer les doublons)
- COUNT(DISTINCT attribut) compte le nombre d'attributs sans doublons
- COUNT(DISTINCT *) est invalide

Count

Afficher le nombre de salariés qui ont un salaire fixe

```
SELECT COUNT(*)
FROM       Employé
WHERE      Salaire IS NOT NULL
```

Afficher le nombre d'employés qui ne gagnent pas le même salaire

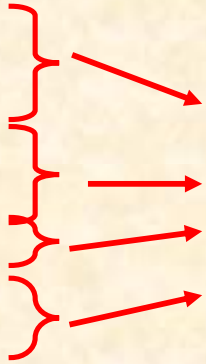
```
SELECT COUNT(DISTINCT Salaire)
FROM   Employé
```

Regroupements

Afficher le nombre de *Commandes* de chaque *Client* (qui a passé au moins une *Commande)*

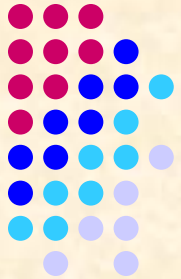
```
SELECT     noClient, COUNT(*) AS nombreCommandes
FROM       Commande
GROUP BY   noClient
```

Table *Commande*		
noCommande	**dateCommande**	**noClient**
1	01/06/2000	10
3	02/06/2000	10
4	05/07/2000	10
2	02/06/2000	20
6	09/07/2000	20
5	09/07/2000	30
7	15/07/2000	40
8	15/07/2000	40



noClient	nombreCommandes
10	3
20	2
30	1
40	2

Subtilités des Regroupements

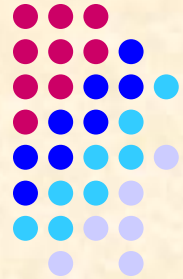


```
SELECT     noClient, NomClient AS nombreCommandes
FROM       Commande
GROUP BY   noClient
```

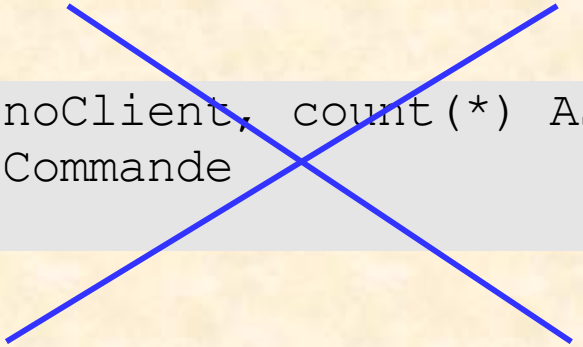
```
SELECT     noClient, NomClient
FROM       Commande
GROUP BY   noClient, NomClient
```

```
SELECT     noClient, COUNT(*) AS nombreCommandes
FROM       Commande
GROUP BY   noClient
```

Subtilités des Regroupements



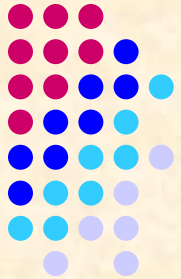
```
SELECT     noClient, count(*) AS nombreCommandes
FROM       Commande
```



```
SELECT     noClient, Count(*) AS nombreCommandes
FROM       Commande
GROUP BY   noClient
```

```
SELECT     COUNT(*) AS nombreCommandes
FROM       Commande
```

Subtilités des Regroupements

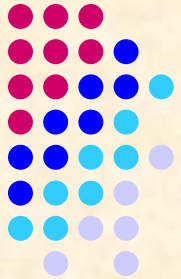


```
SELECT     noClient, NomClient
FROM       Commande
GROUP BY   noClient
```

```
SELECT     NomClient
FROM       Commande
GROUP BY   noClient, NomClient
```

```
SELECT     NomClient
FROM       Commande
GROUP BY NomClient
```

Regroupements : Clause HAVING



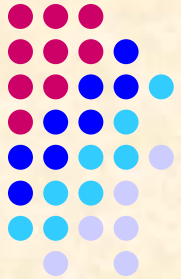
Afficher le nombre de *Commandes* de chaque *Client* qui a passé deux *Commande*s ou plus

```
SELECT     noClient, COUNT(*) AS nombreCommandes
FROM       Commande
GROUP BY   noClient
WHERE COUNT(*) >= 2
```

La clause WHERE ne peut pas être utilisée après GROUP BY

```
SELECT     noClient, COUNT(*) AS nombreCommandes
FROM       Commande
GROUP BY   noClient
HAVING     COUNT(*) >= 2
```

Regroupements : Clause HAVING



Afficher le nombre de *Commandes* de chaque *Client* passées après le 02/06/2000

```
SELECT     noClient, COUNT(*) AS nombreCommandes
FROM       Commande
WHERE      dateCommande > '02/06/2000'
GROUP BY   noClient
```

Afficher le nombre de *Commandes* de chaque *Client* qui a passé deux Commandes ou plus après le 02/06/2000

```
SELECT     noClient, COUNT(*) AS nombreCommandes
FROM       Commande
WHERE      dateCommande > '02/06/2000'
GROUP BY   noClient
HAVING     COUNT(*) >= 2
```

Requêtes imbriquées

Afficher toutes les informations concernant les *Clients* qui ont passé au moins une *Commande*

```
SELECT    Client.noClient, nomClient, noTéléphone
FROM      Client, Commande
WHERE     Client.noClient = Commande.noClient
```

```
SELECT    noClient, nomClient, noTéléphone
FROM      Client
WHERE     noClient IN
        (SELECT   noClient
         FROM     Commande)
```

Requêtes imbriquées

```
SELECT      *
FROM        Commande
WHERE       noClient =
     (SELECT      noClient
      FROM        Client
      WHERE       nomClient = 'Chbeir')
```

Valide car une seule ligne renvoyée par la sous-requête

```
SELECT      *
FROM        Client
WHERE       0 <
    (SELECT      COUNT(*)
     FROM        Commande
     WHERE       noClient = Client.noClient)
```

Référence à une colonne non locale

Requêtes imbriquées

```
SELECT noClient, dateCommande
FROM
    (SELECT *
     FROM Commande
     WHERE       dateCommande > '05/07/2000'
    )
```

SQL 2 !

Conditions ALL, ANY

Afficher les employés qui touchent un salaire supérieur à n'importe quel autre employé (le plus élevé)

```
SELECT     *
FROM       Employé
WHERE      Salaire >= ALL (
                          SELECT     Salaire
                          FROM       Employé)
```

Afficher les employés qui touchent un salaire supérieur à un employé quelconque dont le nom commence par c

```
SELECT     *
FROM       Employé
WHERE      Salaire > ANY (
                          SELECT     Salaire
                          FROM       Employé
                          WHERE Nom_employe like "c*")
```

EXISTS, NOT EXISTS

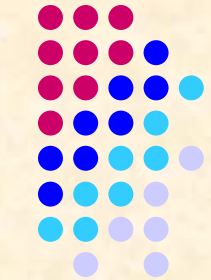
Afficher les *Clients* qui ont passé au moins une *Commande*

```
SELECT     *
FROM       Client
WHERE      EXISTS
    (SELECT     *
     FROM       Commande
     WHERE      noClient = Client.noClient)
```

Afficher les *Clients* qui ne sont pas en même temps des *employés*

```
SELECT     *
FROM       Client e
WHERE      NOT EXISTS
    (SELECT     *
     FROM       Employé
     WHERE      N°SS_Employé = e.N°SS)
```

Exprimer une division algébrique avec count()



Clients(n°client,nomclient,preclient)

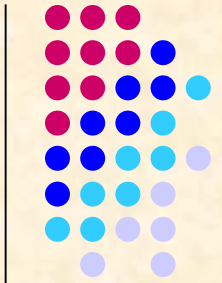
Commande(n°client,n°commande,n°produit,quantité,PU)

Produits(n°produit,stock,PU)

Quels sont les clients qui ont commandé tous les produits ?

- Solution avec count() :
  - ⇨ Compter le nombre de produits différents
  - ⇨ Compter **pour chaque client** le nombre de produits différents commandés
  - ⇨ Comparer les deux résultats

Exprimer une division algébrique



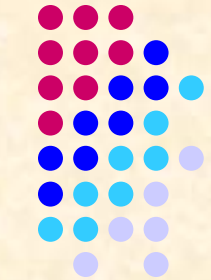
Select n°client from Commandes

Group by n°client

Having count(distinct n°produit) =

Select count (*) from Produits

Exprimer une division relationnelle avec EXISTS



Clients(n°client,nomclient,preclient)

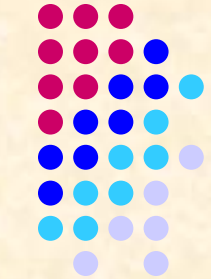
Commande(n°client,n°commande,n°produit,quantité,PU)

Produits(n°produit,stock,PU)

Quels sont les clients qui ont commandé tous les produits ?

- Solution avec EXISTS :
  - ⇨ Sélectionner les clients tels que
  - ⇨ Il n'existe pas de produit qui n'ait pas été commandé

Exprimer une division relationnelle avec EXISTS

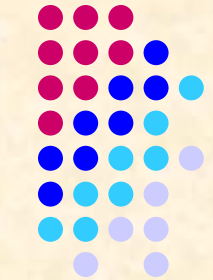


- ⇨ Sélectionner les clients tels qu'il n'existe pas de produit qui n'ait pas été commandé

Équivaut à

- ⇨ Sélectionner les clients c tels que
- ⇨ Il n'existe pas de produit p tel que
- ⇨ Il n'existe pas de ligne de commande pour le produit p et le client c

Exprimer une division relationnelle avec EXISTS



```
Select n°client c from Client
Where not exists
(Select n°produit p from Produits
Where not exists
(Select * from Commandes where n°produit=p
   and n°client=c))
```

Modification de données

- Insérer des données

```
INSERT INTO Nom_de_la_table(colonne1,colonne2,colonne3,...)
VALUES (Valeur1,Valeur2,Valeur3,...)
```

```
INSERT INTO Nom_de_la_table(colonne1,colonne2,...)
SELECT colonne1,colonne2,...
FROM Nom_de_la_table2
WHERE qualification
```

Modification de données

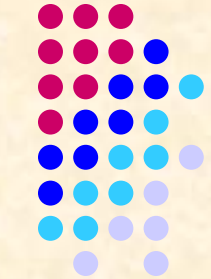
- Modifier des lignes de données

```
UPDATE Nom_de_la_table
SET Colonne = Valeur_Ou_Expression
WHERE qualification
```

- Supprimer des données

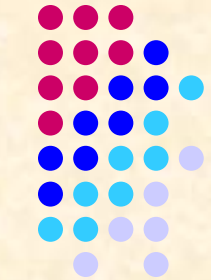
```
DELETE FROM Nom_de_la_table
WHERE qualification
```

Langage de Définition des Données (LDD)



- Création de base de données
- Création de Table
  - ⇨ Table simple
  - ⇨ Contraintes d'intégrité
  - ⇨ Clé primaire
  - ⇨ Intégrité référentielle
- Modification de structure d'une table
- Suppression d'une table

Création de Base de Données



- Norme SQL :

CREATE SCHEMA [nom_schema]

[AUTHORIZATION utilisateur]

[DEFAULT CHARACTER SET jeu_caractères] [liste_des_objets_du_schéma]

TABLES

Peu implémenté dans les SGBDR → CREATE DATABASE

Création de table

- La plus simple:

```
CREATE TABLE CLIENT (
CLI_NOM VARCHAR(32),
CLI_PRENOM VARCHAR(32)
)
```

Contrainte NULL / NOT NULL

- On précise NOT NULL pour indiquer un champ qui doit obligatoirement être renseigné

```
CREATE TABLE PERSONNE (
ID INTEGER NOT NULL,
NOM VARCHAR(32) NOT NULL,
PRENOM VARCHAR(32)  NULL,   DATE_NAISSANCE
DATE)
```

Optionnel

Valeur par défaut

```
CREATE TABLE PERSONNE (
ID INTEGER NOT NULL,
NOM VARCHAR(32) NOT NULL,
PRENOM VARCHAR(32)  NULL,
SEXE CHAR(1) DEFAULT 'M',
     DATE_NAISSANCE   DATE DEFAULT CURRENT_DATE)
```

Fonction dépendante du SGBD

Il est interdit de spécifier comme valeur par défaut le résultat d'une requête

Unicité

```
CREATE TABLE PERSONNE (
ID INTEGER NOT NULL UNIQUE,
NOM VARCHAR(32) NOT NULL,
PRENOM VARCHAR(32)  NULL,    DATE_NAISSANCE
DATE)
```

Selon le SGBD, NOT NULL avant UNIQUE est obligatoire ou interdit ! ☺

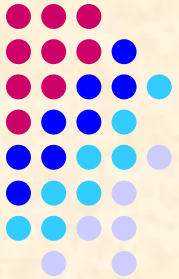
Validation

On peut spécifier des contraintes de validation très complexes

```
CREATE TABLE PERSONNE (
ID INTEGER CHECK (VALUE>0),
NOM VARCHAR(32) CHECK (CHARACTER_LENGTH(VALUE)>2),
PRENOM VARCHAR(32),
SEXE CHAR(1) CHECK (VALUE IN ('M','F')),
DEPT VARCHAR(32) CHECK (VALUE IN (SELECT NOM FROM
DEPARTEMENT) OR IS NULL)
)
```

SELECT est autorisé

Validation globale



CONSTRAINT nom_contrainte CHECK (prédicat)

```
CREATE TABLE PERSONNE (
ID INTEGER,
NOM VARCHAR(32) CHECK (CHARACTER_LENGTH(VALUE)>2),
PRENOM VARCHAR(32),
SEXE CHAR(1) CHECK (VALUE IN ('M','F')),
DEPT VARCHAR(32)
CONSTRAINT CK_PRS CHECK ((ID>0) AND (DEPT IN (SELECT NOM FROM
DEPARTEMENT) OR IS NULL))

)
```

Clé primaire

- Simple

```
CREATE TABLE PERSONNE (
ID INTEGER NOT NULL PRIMARY KEY,
NOM VARCHAR(32) NOT NULL,
PRENOM VARCHAR(32)  NULL,    DATE_NAISSANCE
DATE)
```

- Composée

```
CONSTRAINT nom_contrainte PRIMARY KEY (liste_colonne)
```

```
CREATE TABLE PERSONNE (
NOM VARCHAR(32) NOT NULL,
PRENOM VARCHAR(32)  NULL,    DATE_NAISSANCE
DATE
CONSTRAINT C_PRS PRIMARY KEY (NOM,PRENOM)
)
```

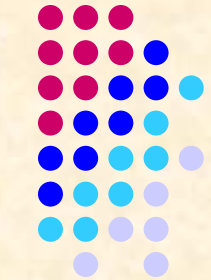
Intégrité référentielle

- FOREIGN KEY permet de définir une contrainte d'intégrité référentielle entre deux tables

```
CREATE TABLE FACTURE (
ID_FACT INTEGER,
ID INTEGER FOREIGN KEY REFERENCES PERSONNE(ID),
DATE DATE
)
```

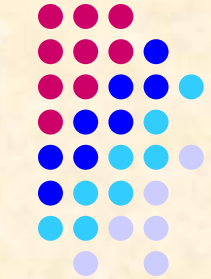
L'utilisation d'un nom identique n'est pas obligatoire, mais très conseillée (natural join)

Intégrité référentielle multi-colonnes



```
CREATE TABLE FACTURE (
ID_FACT INTEGER,
NOM VARCHAR(32),
PRENOM VARCHAR(32),
DATE DATE,
CONSTRAINT C_FACT_PERS FOREIGN KEY (NOM,PRENOM)
REFERENCES PERSONNE(NOM,PRENOM)
)
```

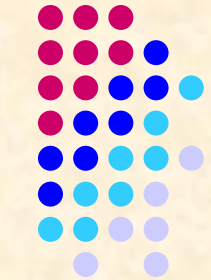
Contraintes d'intégrité référentielle



- Syntaxe complète

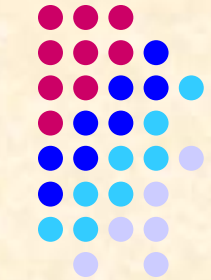
CONSTRAINT nom_contrainte FOREIGN KEY (liste_colonne_table)
REFERENCES table_référencée (liste_colonne_référencées)
[MATCH { FULL | PARTIAL | SIMPLE }]
[{ ON UPDATE { NO ACTION | CASCADE | RESTRICT | SET NULL | SET DEFAULT }]
[{ ON DELETE { NO ACTION | CASCADE | RESTRICT | SET NULL | SET DEFAULT }]
[{ INITIALLY DEFERRED | INITIALLY IMMEDIATE } [[NOT] DEFERRABLE] | [NOT] DEFERRABLE [{ INITIALLY DEFERRED | INITIALLY IMMEDIATE }]]

Modification de la structure



- ALTER permet de :
  - ⇨ supprimer une colonne
  - ⇨ Supprimer une contrainte
  - ⇨ Ajouter une colonne
  - ⇨ Ajouter une contrainte
- ALTER ne permet pas de :
  - ⇨ Changer le nom d'une colonne
  - ⇨ Changer le type d'une colonne

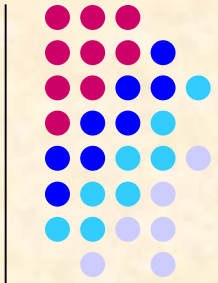
Modification de la structure



- Syntaxe

```
ALTER TABLE nom_table {
   ADD definition_colonne |
   ALTER nom_colonne {
    SET DEFAULT valeur_défaut |
    DROP DEFAULT } |
   DROP nom_colonne [ CASCADE | RESTRICT ] |
   ADD définition_contrainte_ligne |
   DROP nom_contrainte [ CASCADE | RESTRICT ] }
```

Modification de la structure

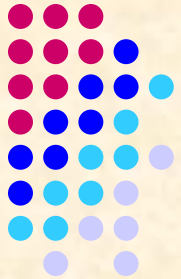


- Comment modifier le nom d’une colonne ?

Supprimer une table

- DROP TABLE nom_table

Langage de Contrôle des Données (LCD)



- Base de Données multi-utilisateur
- But du LCD : contrôler l'accès aux données des différents utilisateurs
- Moyens :
  - ⇨ Définition d'utilisateurs, de groupes d'utilisateurs (dépendant du SGBD)
  - ⇨ Octroi de privilèges (GRANT)
  - ⇨ Révocation de privilèges(REVOKE)

GRANT / REVOKE

- GRANT autorise pour une personne :
  - ⇨ La consultation d'une table
  - ⇨ La modification d'une table
  - ⇨ La transmission de ses droits à un autre utilisateur
- REVOKE fait l'inverse ☺

Gestion Complexe → Travail du DBA (Administrateur de Bases de Données)